COMP4121 Project 6 Proposal: Polymorphic Types

MAKSIMOVICH, Roman | TSZ, Wu Yiu

Our intention is to accomplish the following goals:

- **Rewrite the Amy compiler frontend in Rust.** This will allow us to establish a separate codebase that we are more familiar with, make sure it is clean and consistent, and to revisit the implementation of the frontend.
- **Implement parametric polymorphism for functions and ADT's.** Currently, the Amy language is hard to use, in part because it lacks polymorphism. The `Option` class of the `Std` module only supports integer values, whereas one would often desire a parameterized `Option[A]` type. Similarly, one would desire a `List[A]` construct and a way to write polymorphic functions. We aim to extend Amy with the following syntax:

```
// A class parameterized by type variables A_1, ..., A_n
abstract class T[A_1, A_2, ..., A_n]

// Constructors may take arguments of types A_1, ..., A_n.
case class C1 (a_1: A_1, ..., a_m: A_m) extends T
case class C2 (b_1: A_1, ..., b_k: A_k) extends T

// Funstions may be parametrically polymorphic
def fun[A_1, ..., A_n, A] (a_1: A_1, ..., a_l: A_l): A := ... end fun
```

For example,

```
abstract class List[A]
case class Nil() extends List
case class Cons(h: A, t: List[A]) extends List

def length[A] (lst: List[A]): Int(32) := lst match {
  case Nil() => 0
  case Cons(h, t) => 1 + length(t)
}
end length
```

Values may also have polymorphic types:

```
Nil() : forall A. List[A]
```

Pattern matching on values typed as `forall A. A` will be *disallowed*, since this type yields no information about the value.